A Resource-aware Federated Transfer Learning Framework for Clients with non-IID Data

He Zhang
School of Electronic and Information Engineering
Lanzhou Jiaotong University
Lanzhou, China
20233201246@stu.lzjtu.edu.cn

***Abstract*—Federated learning (FL) enables collaborative model training without sharing raw data, but its performance often degrades significantly under non-IID data distributions and heterogeneous client resources. Existing approaches either focus on improving model accuracy under statistical heterogeneity or rely on complex architectures that are not suitable for resource-constrained devices. To address this issue, this paper proposes a Resource-Aware Federated Transfer Learning (RA-FTL) framework for heterogeneous clients. Specifically, before performing joint data partitioning, we set aside a small, class-balanced proxy validation set from the original public dataset. Subsequently, the remaining samples are partitioned among the clients using a Dirichlet distribution. As a result, the server does not access any client's private local training data. Through a competitive selection mechanism, we first identify a set of clients with higher computational power and larger local datasets, and then train a relatively optimal shared feature extractor based on the proxy validation set. Subsequently, the learned feature layers are transferred to resource-constrained clients, and a portion of the network is frozen on these clients to reduce local training costs and communication overhead. Based on this design, the proposed framework jointly considers client resource disparity, transfer depth, and non-IID data distributions. Experiments on the MSTAR dataset under multiple Dirichlet-based non-IID settings show that RA-FTL achieves faster convergence and lower training overhead than conventional FL, while maintaining competitive classification accuracy. In addition, the effects of freezing depth and the number of pre-training clients are systematically analyzed, revealing the trade-off between transferability, model adaptability, and communication efficiency.**

Keywords—federated learning; non-IID data; resource-aware; transfer learning; communication and computation efficiency

I. INTRODUCTION

With the rapid development of digital technology, smart devices and sensors are continuously generating massive amounts of data [1]. In emerging sensing applications such as intelligent transportation, autonomous monitoring, radar target recognition, and Internet of Things (IoT) systems, sensing devices continuously collect large-scale environmental and target information for distributed intelligent analysis. In particular, communication-based perception and Integrated Sensing and Communication (ISAC) systems require collaborative model training among geographically dispersed devices while maintaining local data privacy. Traditional centralized machine learning typically requires uploading raw data to cloud servers for model training, raising serious concerns regarding privacy breaches, data ownership, and communication overhead. Federated learning (FL) has emerged as a promising distributed learning paradigm to address these issues by enabling multiple clients to collaboratively train a global model without sharing their raw data [2]. Instead, only model updates are exchanged between clients and the server, which significantly improves privacy preservation and reduces direct data transmission risk.

Despite these advantages, the performance of FL is often severely degraded in practical deployments due to statistical and system heterogeneity. On the one hand, client data are commonly non-independent and identically distributed (non-IID) [3], which leads to inconsistent local update directions and unstable global convergence. On the other hand, clients usually have different computation capabilities, storage conditions, and communication resources [4]. Under such heterogeneous environments, applying the same training workload to all clients is often inefficient or even impractical, especially for resource-constrained devices. Therefore, a key challenge is how to maintain satisfactory model accuracy while reducing computation and communication overhead of FL under non-IID data and heterogeneous client resources.

Existing studies have attempted to improve FL from different perspectives, such as robust optimization under non-IID data, communication-efficient aggregation, and transfer learning-based model adaptation [5]. However, many existing methods either focus mainly on statistical heterogeneity while overlooking resource disparity across clients or rely on relatively complex training strategies that may not be suitable for clients with limited computation capability. In contrast, federated transfer learning (FTL) [6] provides a lightweight and practical alternative. By separating the model into shared feature extraction layers and task-specific layers, FTL can transfer useful representations learned during pre-training and reduce the number of parameters that need to be frequently updated and transmitted. This makes FTL particularly attractive for heterogeneous FL scenarios, where some clients can undertake heavier pre-training workloads while other clients require lower local training complexity.

Motivated by this observation, this paper proposes a Resource-Aware Federated Transfer Learning (RA-FTL) framework for non-IID data and heterogeneous clients. The core idea is to first evaluate clients according to their computation capability and local data scale, and then select a subset of high-resource clients to perform pre-training for the shared feature

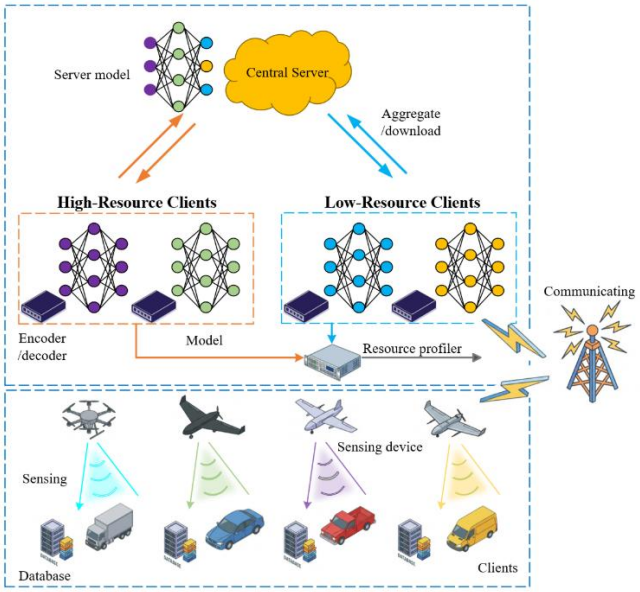

Fig. 1 System Model Architecture.

To validate the effectiveness of the proposed framework, experiments are conducted on the public MSTAR dataset under multiple Dirichlet-based non-IID settings. The study systematically compares conventional FL and FTL-based training strategies and further investigates the influence of freezing depth and the number of pre-training clients on model performance. Experimental results show that the proposed RA-FTL framework can effectively reduce computation and communication overhead while maintaining competitive classification accuracy, demonstrating its potential for federated sensing tasks with heterogeneous client resources. The main contributions of this paper are summarized as follows:

- A resource-aware federated transfer learning framework is proposed for FL systems with non-IID data and heterogeneous client resources.
- A client selection and transfer mechanism is designed, where high-resource clients are used for pre-training and the learned feature extractor is migrated to resource-constrained clients.
- Extensive experiments on a public sensing dataset are conducted to analyze the effects of data heterogeneity, freezing depth, and the number of pre-training clients, verifying the efficiency-accuracy trade-off of the proposed method.

II. Related Work

At present, the development of federated learning still faces several major challenges. Communication efficiency is a critical bottleneck in large-scale distributed environments, where high latency and limited bandwidth can significantly slow down model convergence [7]. In addition, FL systems are inherently affected by both statistical heterogeneity and system heterogeneity. In practice, different clients usually rely on different hardware platforms and store their data independently, making unified collaborative training difficult and aggravating the data-silo problem. At the same time, clients often exhibit substantial differences in computational capability and resource availability.

In terms of communication efficiency, McMahan et al. proposed the FedAvg algorithm to reduce communication frequency by allowing clients to perform multiple local updates before global aggregation, and this method has become the standard baseline in federated learning [2]. To mitigate communication bottlenecks in federated learning, Chen et al. proposed a communication-efficient FL method with layerwise asynchronous model update and temporally weighted aggregation, where shallow layers are updated more frequently than deep layers to reduce communication overhead while maintaining model performance [8]. Chuhan Wu et al. proposed FedKD, which combines knowledge distillation with SVD-based gradient compression to reduce communication overhead in federated learning. Experimental results show that FedKD can reduce communication costs by up to 94.89% while maintaining competitive performance [9]. Sa Xiao et al. proposed DB-FL, a DAG blockchain-enabled federated dropout learning framework that combines semi-asynchronous training and resource allocation to reduce network delay while improving model generalization [10].

In terms of heterogeneity, existing studies mainly consider both statistical heterogeneity and system heterogeneity. To address model drift caused by non-IID data, Li Tian et al. proposed FedProx [11], which introduces a proximal term to constrain local model updates and improve optimization stability under heterogeneous data distributions. Federated transfer learning has also been explored under different levels of non-IID data, where multi-round transfer is used to improve robustness and mitigate forgetting [12]. Sattler et al. proposed Sparse Ternary Compression (STC) to address communication inefficiency under non-IID data and high-dimensional parameter transmission, combining Top-K sparsification with Golomb coding [13]. Their method significantly reduces communication overhead and remains robust under non-IID settings.

III. System Model

The system model is illustrated in Fig. 1, which mainly consists of four parts. In the first part, sensors and IoT devices are responsible for sensing and observing target objects, and the collected data are stored locally for subsequent model training. These devices are regarded as the clients in the federated learning system. In the second part, after data collection, each client performs local computation and model training based on its own dataset. In the third part, the clients communicate with the central server through wireless links to transmit model-related information. In the fourth part, the overall federated learning architecture is formed, where the server coordinates global aggregation and broadcasts the updated model to all participating clients.

A. Federated Learning Models

Federated learning (FL) is a typical distributed learning framework consisting of one central server and multiple clients. Its main idea is to complete collaborative model training without sharing raw data, where only model parameters are exchanged for global aggregation. Suppose that the system contains K

clients, and the local dataset of client k is denoted by $\mathcal{D}_k$, where each sample contains a two-dimensional SAR image and its corresponding target label. The local loss function of client $k$can be written as

$$F_k(\mathbf{w}) = \frac{1}{|\mathcal{D}_k|} \sum_{(x,y)\in\mathcal{D}_k} \ell(\mathbf{w}; x, y). \tag{1}$$

At communication round t, the server sends the global model $\mathbf{w}^{(t)}$ to each client. Then, each client updates the model using its local dataset. After local training, the server collects the uploaded parameters from all participating clients and performs weighted aggregation to obtain the updated global model, i.e.,

$$\mathbf{w}^{(t+1)} = \sum_{k=1}^{K} \alpha_k \, \mathbf{w}_k^{(t+1)}, \tag{2}$$

where $\alpha_k$denotes the aggregation weight of client k. In general, $\alpha_k$can be determined according to the amount of local data or other factors such as computation capability. This process is repeated until the model converges.

B. SENSING MODEL

The data used in this paper are derived from a constructed distributed SAR sensing model. As an active microwave imaging radar, synthetic aperture radar (SAR) transmits modulated pulses and receives the echoes reflected from the target, thereby achieving high-resolution two-dimensional imaging. The transmitted linear frequency modulated pulse can be expressed as

$$s(\tau) = \mathrm{rect}\left(\frac{\tau}{T_p}\right)\cos(2\pi f_c\tau + \pi K_r\tau^2), \tag{3}$$

where T_p denotes the pulse width, f_c is the carrier frequency, and K_r is the chirp rate. Let η denote the slow-time variable generated by platform motion and let $R(\eta)$ represent the instantaneous slant range variation of the target with respect to slow time. Then, the received echo signal can be written as

$$r(\tau, \eta) = A\, s\left(\tau - \frac{2R(\eta)}{c}\right), \tag{4}$$

where A denotes the echo amplitude and c is the speed of light.

In the experimental part, the public MSTAR (Moving and Stationary Target Acquisition and Recognition) [14] dataset is adopted for validation. This dataset, released by the U.S. Defense Advanced Research Projects Agency (DARPA), contains 128×128 pixel SAR images of multiple classes of ground military targets, and is widely used in SAR target recognition research due to its high image quality and clear target features. To simulate a practical distributed sensing network environment, the dataset is partitioned across multiple clients in a non-IID manner, denoted by $\mathcal{D}_k$, thereby constructing a realistic federated sensing and training scenario.

C. COMMUNICATION MODEL

In communication-assisted sensing scenarios, the performance of federated learning depends not only on local data quality and local model training, but also on wireless channel conditions and the adopted coding scheme. A typical communication process generally consists of three stages: model compression and encoding, channel transmission, and server-side decoding and aggregation. To reduce bandwidth consumption, each client first compresses and encodes its local model parameters as

$$e_k^{(t)} = \phi_k\left(\omega_k^{(t)}\right), \tag{5}$$

where ϕ_k denotes the source coding function. The compressed result is then mapped to the channel input signal as

$$x_k^{(t)} = \varphi_k\left(e_k^{(t)}\right), \tag{6}$$

where $\varphi_k(\cdot)$ represents the channel mapping function.

After wireless transmission, the received signal at the server can be expressed as

$$y_k^{(t)} \sim P\left(y_k^{(t)} \mid x_k^{(t)}\right), \tag{7}$$

where $P(\cdot)$ denotes the conditional probability distribution of the received signal, which characterizes the effects of channel fading and noise. After decoding the received signal, the server obtains an estimate of the recovered model parameters as

$$\widehat{\omega}_k^{(t)} = \phi_k^{-1}\left(\varphi_k^{-1}\left(y_k^{(t)}\right)\right). \tag{8}$$

Finally, the server performs weighted aggregation over all recovered local models, i.e.,

$$\omega^{(t+1)} = \sum_{k=1}^{K} \alpha_k \, \widehat{\omega}_k^{(t)}, \tag{9}$$

where $\alpha_k$denotes the aggregation weight of client k.

In FTL mode, communication overhead can be divided into the pre-training phase and the main federated training phase. Assume that the model contains F layers, and the number of parameters in the f-th layer is I_f. If each parameter occupies $d$bits the pre-training phase runs for T_{pre} rounds and the main training phase runs for T_{main} rounds, and the number of frozen layers is L, then the total overhead of uplink communication can be expressed as

$$U_{FTL} = dK\left(T_{pre}\sum_{f=1}^{L} I_f + T_{main}\sum_{f=L+1}^{F} I_f\right) \tag{10}$$

D. DATA DISTRIBUTION MODEL

In distributed sensing scenarios, each client independently collects local data, and the resulting data distributions usually do not satisfy the independent and identically distributed (IID) assumption. Let the data distribution of client k be denoted by $P_k(x, y)$. Then, for different clients, the local data distributions generally satisfy

$$P_k(x, y) \neq P_j(x, y), k \neq j. \tag{11}$$

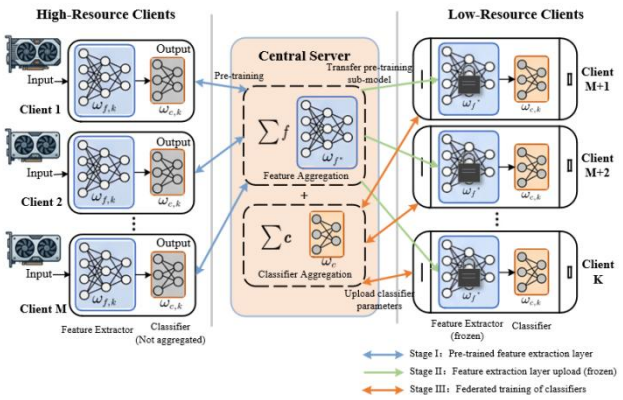

Fig. 2 RA-FTL Model Architecture.

To characterize class imbalance across clients, the Dirichlet distribution is adopted to model the class proportion of each client, i.e.,

$$\pi_k \sim \mathrm{Dir}(\alpha), \tag{12}$$

where α is the concentration parameter [15]. A smaller value of αindicates a more severely imbalanced and heterogeneous data distribution across clients.

IV. Resource-aware Federated Transfer Learning Algorithm

A. Federated Transfer Learning Model

In scenarios where the number of pre-training samples M is limited, we propose a method that combines training, comparison, and competitive selection to optimize the pre-training process. This method calculates an evaluation coefficient based on the client's data volume D_k and computational capacity C_k, thereby screening all possible groups of pre-trained clients. Subsequently, each client undergoes training simultaneously, and the group S with the highest accuracy on the proxy validation set is selected to serve as the frozen parameters for transfer learning.

$$S \subseteq \{1,2,\dots,N\}, \qquad |S| = M; \tag{13}$$

The comprehensive evaluation coefficients for the resource-aware model are as follows:

$$R_k = \frac{D_k}{\sum_{k=1}^{K} D_k} + \frac{C_k}{\sum_{k=1}^{K} C_k}, \qquad R \subseteq (0\,,2). \tag{14}$$

Thus, based on the R_k evaluation scores, high-resource clients are selected to serve as high-quality data sources for training the feature extraction submodels.

In federated transfer learning, deep neural networks $\omega_k^t$are divided into feature extraction layers $\omega_{k,1}^t$ and classification layers $\omega_{k,2}^t$. Based on pre-computed resource-aware coefficients, we select clients with suitable datasets for pre-training. We then select the best-performing feature extraction layers—those with high data coverage—from each group for synchronous training. $f_{\omega_{k,1}}^{(1)}$. At this stage, we do not concern ourselves with the parameters of classification layers. After broadcasting the feature extraction layers $f_{\omega_{k,2}}^{(2)}$, we proceed with joint training in collaboration with clients with limited resources. At this point, RA-FTL can be expressed as

$$f_{\omega_k} = \left\{ f_{\omega_k}^{(1)}, f_{\omega_k}^{(2)} \right\}. \tag{15}$$

Algorithm 1 Resource-aware FTL

Input: Dataset D_k and Computational quantity C_k

Output: Aggregated model $\sum_{k=1}^{K} \alpha_k \mathbf{w}_k$.

Calculate evaluation coefficient R_k.
Select the high resource M clients based on R_k.
Initialize sub-model f_{ω_k} and learning rate r.
Divide f_{ω_k} into sub-model $f_{\omega_k}^{(1)}$ and sub-model $f_{\omega_k}^{(2)}$.
Pre-train sub-model $f_{\omega_k}^{(1)}$ in **FedAvg** $\mathrm{F}(f_{\omega_k}^{(1)}, M, T)$.
Copy pre-trained sub-model $f_{\omega_k}^{(1)}$ to other $K - M$ clients
Aggregate sub-model $f_{\omega_k}^{(2)}$ in **FedAvg** $\mathrm{F}(f_{\omega_k}^{(2)}, K - M, T)$.

FedAvg $\mathrm{F}(f_{\omega_k}, K, T)$:
  for each round $t = 1, 2, \dots, T$:
    Aggregate sub-model: $f_{\omega_t} = \sum_{k=1}^{K} \alpha_k \mathbf{w}_{k,t}$
    Broadcast sub-model f_{ω_t} to each client k.
  for each round $t = 1, 2, \dots, T$:
    Update sub-model $f_{\omega_{k,t}}$ and upload $f_{\omega_{k,t}}$ to server.

In addition, we use a windowing approach to determine the convergence of the iteration; specifically, we select a fixed window size ω and calculate the average every ω rounds.

$$\bar{a}_r = \frac{1}{\omega} \sum_{i=r-\omega+1(i\geq 0)}^{r} a_{i,r} \tag{16}$$

Then, using the ω, the mean and variance within that window are calculated at each round. A variance threshold τ is determined based on empirical data; if the variance of the window at a given round $r(r \geq \omega)$ is less than τ, that round is considered to have converged.

$$\mu_r = \frac{1}{\omega} \sum_{k=r-\omega+1}^{r} \bar{a}_r \tag{17}$$

$$\sigma^2 = \frac{1}{\omega} \sum_{k=r-\omega+1}^{r} (\bar{a}_k - \mu_r)^2 \tag{18}$$

This method effectively avoids convergence judgment errors caused by parameter fluctuations. The parameter settings are shown in the table below.

TABLE I
SLIDING WINDOW AND VARIANCE THRESHOLD SETTINGS

α	Window size	Threshold variance
0.1	10	0.5
0.5	8	2.0
1	8	2.0
5	10	2.0

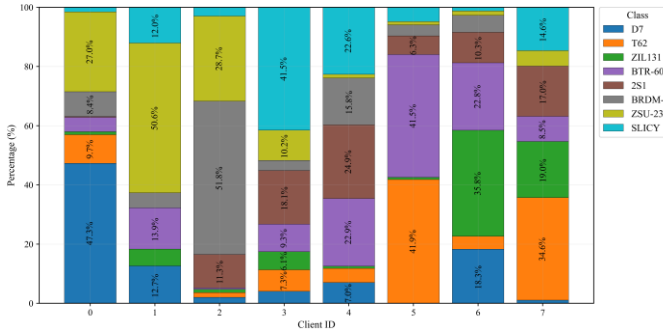

Fig. 3 Distribution of the client dataset when α =0.1.

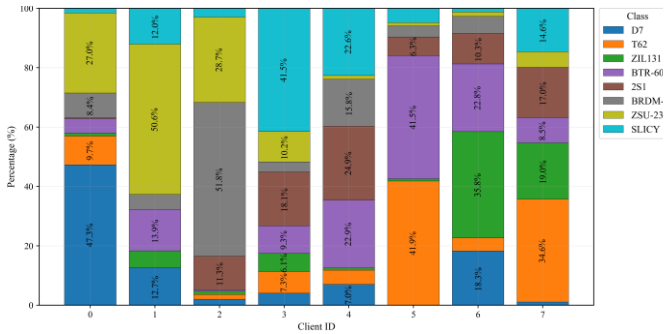

Fig. 4 Distribution of the client dataset when α =0.5.

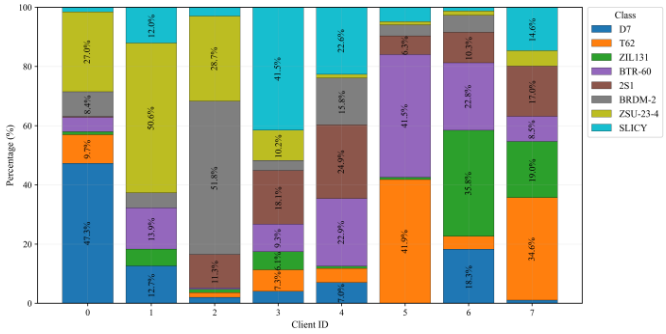

Fig. 5 Distribution of the client dataset when α =1.

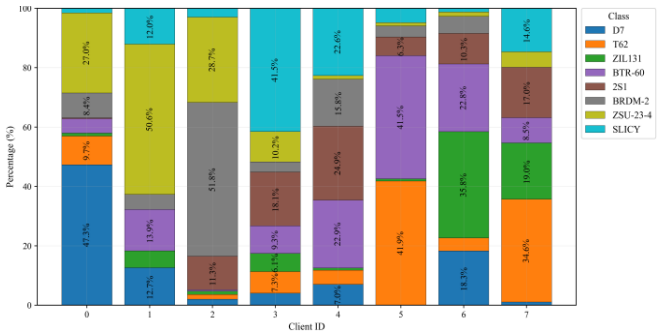

Fig. 6 Distribution of the client dataset when α =5.

V. Experimental Results and Analysis

A. Experimental Setup

This study conducted experiments on an NVIDIA RTX 4090D GPU and presents simulation results for the FedAvg and FTL algorithms. We selected 4,458 samples with resolutions of 15° and 17° from the publicly available MSTAR dataset, first setting aside a small, class-balanced validation set of 200 samples. Of the remaining 4,258 data points, 2,841 were used for training and 1,417 for testing. Based on the Dirichlet distribution, the data was further divided into four distribution scenarios to characterize varying degrees of non-independent and identically distributed (NIID) conditions. The model used by the client is a convolutional neural network consisting of two convolutional layers and two fully connected layers. The batch size was set to 1. An SGD optimizer with a learning rate of 0.00015 was employed, with both pre-training and main training consisting of 100 iterations. Although MSTAR is a widely used benchmark for SAR target recognition, it is a single-domain dataset with relatively limited semantic diversity. Therefore, the Dirichlet-based partition in this work should be interpreted as a controlled simulation of label-distribution skew in distributed SAR sensing scenarios, rather than a complete representation of all real-world heterogeneous data distributions. Extending the evaluation to multi-domain datasets, such as medical images, remote sensing images from different sensors, or user behavior data, will be considered in future work.

B. Data Distribution

To simulate a realistic federated computing scenario, we use a Dirichlet distribution with a concentration parameter α to vary the uniformity of label types, and random combinations to vary the amount of data for each client, thereby modeling a non-independent and identically distributed scenario with K = 8 clients. When the centrality parameter α is 0.1, 0.5, 1, and 5, respectively, the distribution of data types is as follows:

C. Federated Learning

In the experiment, I conducted a comparative analysis of the performance of the FL and FTL models under different data distribution scenarios. As shown in Figures 7 – 8, FTL outperforms FL overall. In a strongly non-IID scenario with α = 0.1, as the number of training iterations increases, FTL initially outperforms FL but eventually falls behind; however, in scenarios with weaker heterogeneity, FTL converges significantly faster and achieves higher accuracy, with an average accuracy of up to 97.9%. This indicates that the transfer advantage is more pronounced in scenarios that tend toward independent and identically distributed (IID) data.

As shown in Figures 9 – 12, experimental data indicate that the impact of the number of pre-trained clients M, on FTL performance is not simply positively correlated; rather, it depends on the scale of the pre-training data and the representativeness of the data distribution. Specifically, when the data is clearly non-independent and non-identically distributed (non-IID), a larger number of pre-trained clients leads to a more significant performance improvement; however, when the data is nearly independent and identically distributed (IID), even a small number of pre-trained clients can provide effective initialization, and the benefits of increasing M are very limited in this case. This suggests that a larger value of M is not necessarily better. Instead, there is an optimal range, and in this experiment, a pre-trained client count of 2 was found to be optimal overall.

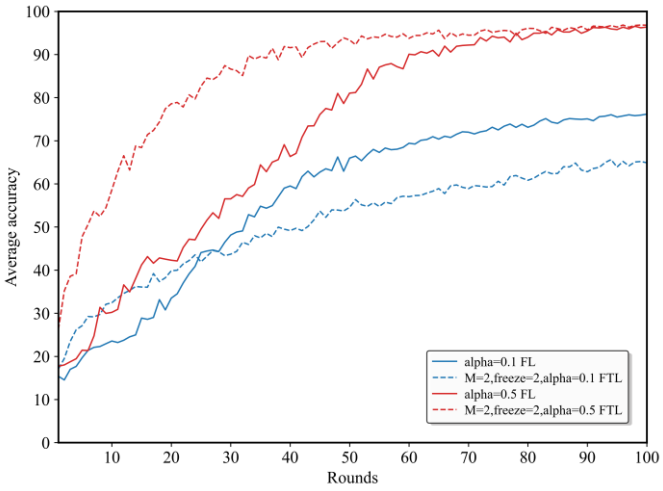

Fig. 7 FL vs FTL under Strong non-IID levels.

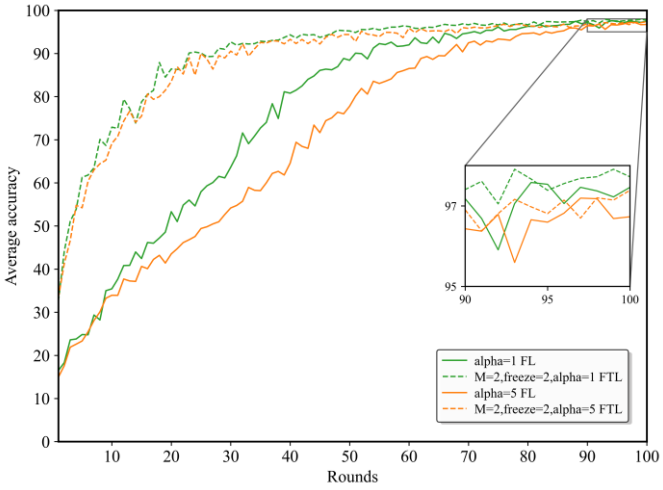

Fig. 8 FL vs FTL under weak non-IID levels.

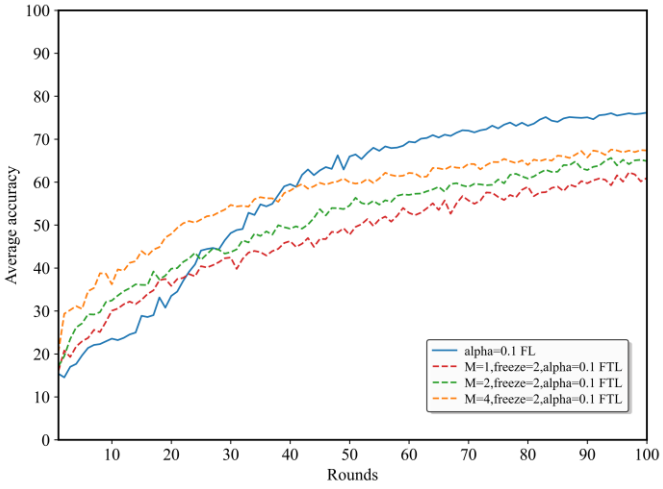

Fig. 9 The effect of the number of training clients M with $\alpha = 0.1$.

As shown in the experimental data, under low heterogeneity, the model performs better as the number of frozen layers increases, particularly during pre-training with a small number of samples. As illustrated in Figure 13-16, when the number of pre-training iterations is 2, the strategy of freezing all layers yields a higher average accuracy and faster convergence rate

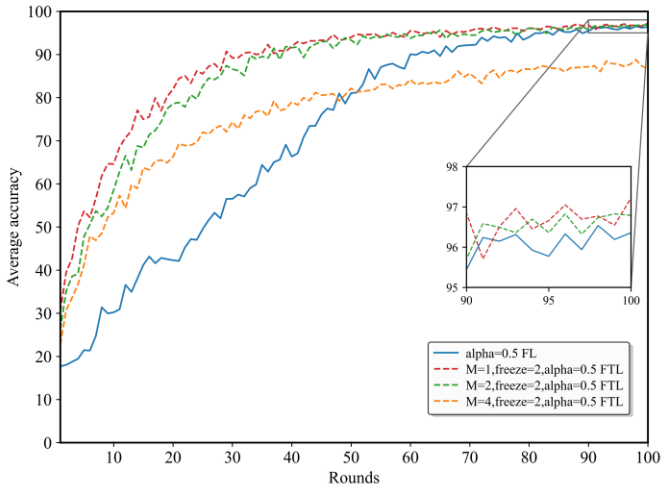

Fig. 10 The effect of the number of training clients M with $\alpha = 0.5$.

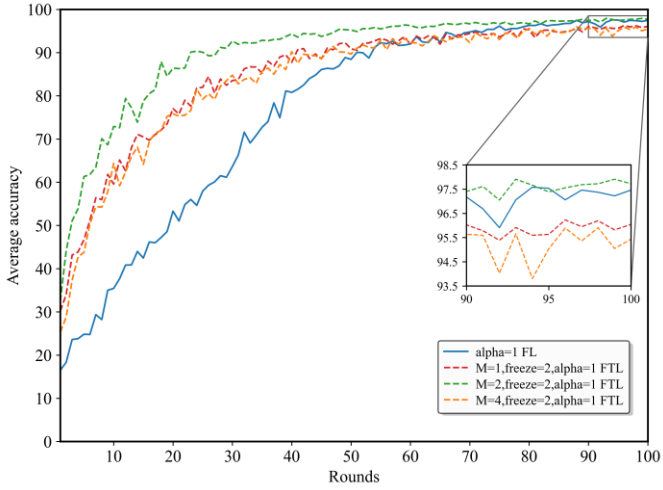

Fig. 11 The effect of the number of training clients M with $\alpha = 1$.

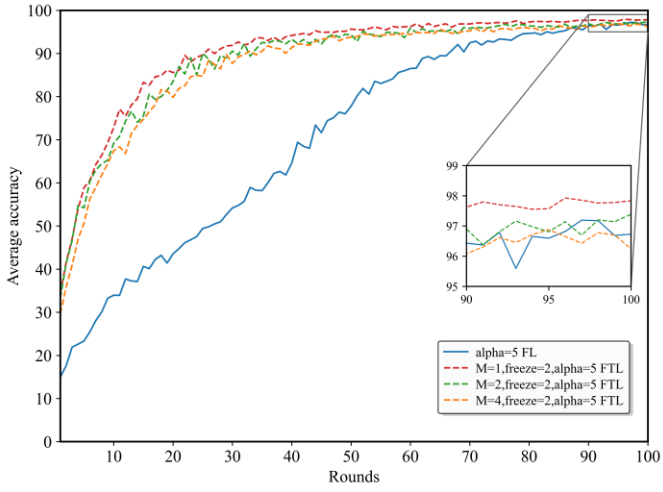

Fig. 12 The effect of the number of training clients M with $\alpha = 5$.

compared to freezing a single layer. However, due to the heterogeneity of client data, freezing too many layers and pre-training many clients results in a lower average accuracy. This

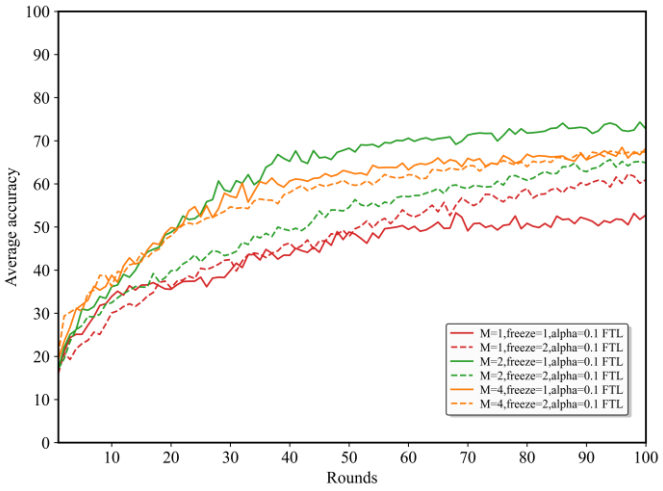

Fig. 13 The effect of freezing depth in FTL at $\alpha = 0.1$.

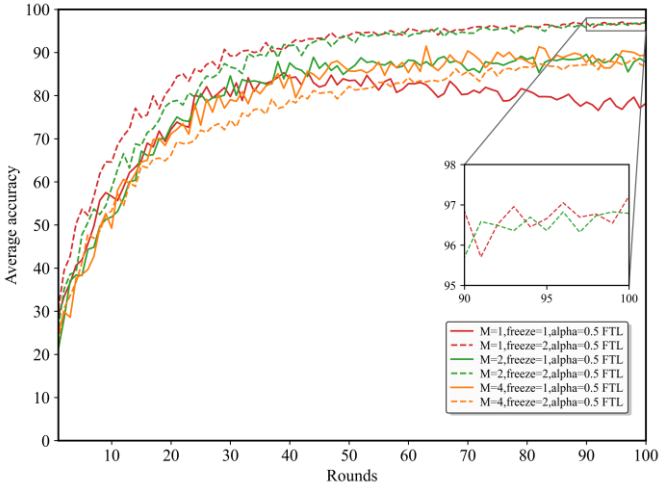

Fig. 14 The effect of freezing depth in FTL at $\alpha = 0.5$.

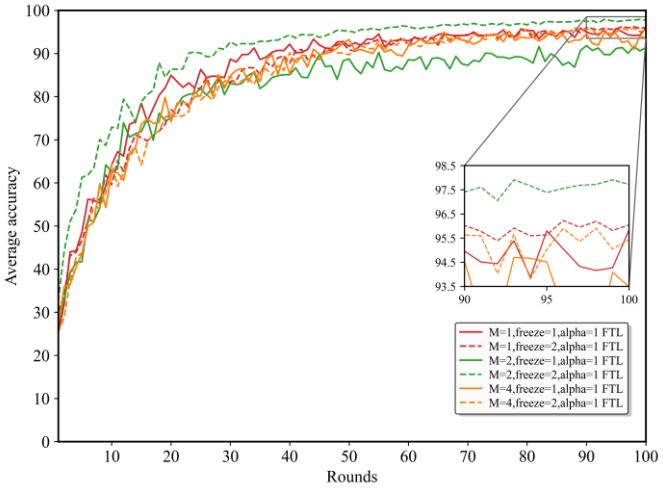

Fig. 15 The effect of freezing depth in FTL at $\alpha = 1$.

discrepancy is particularly pronounced when the number of pre-training iterations is 2; at $\alpha = 1$, the difference in the maximum converged average accuracy between the single-layer and full-layer freezing strategies can reach 10.5%.The results under α=0.1 also reveal a limitation of aggressive layer freezing. When the local label distributions are extremely skewed, the pre-trained feature extractor may be biased toward the selected pre-training clients. In this case, freezing too many layers restricts the ability of resource-constrained clients to adapt to their local distributions, which may reduce final accuracy. Therefore, RA-FTL is more suitable for mild-to-moderate non-IID settings, while shallow freezing is preferred under strong non-IID conditions.

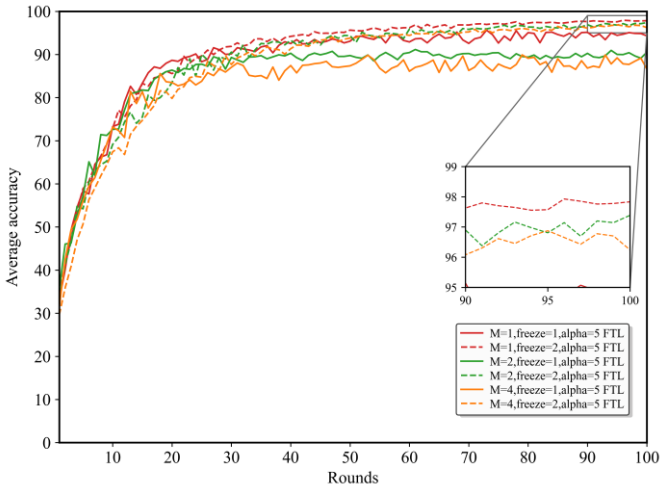

Fig. 16 The effect of freezing depth in FTL at $\alpha = 5$.

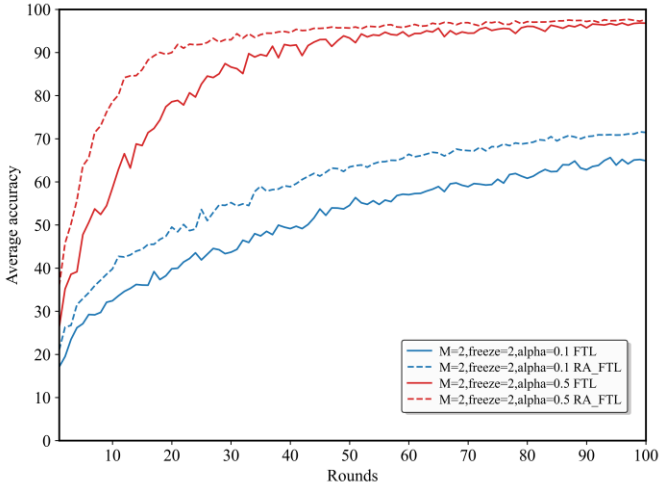

Fig. 17 Comparison between RA-FTL and standard FTL at α =0.1 and 0.5 with freezing 2 layers.

As shown in Figure 17-20 and Table 2-5, under the premise that the number of pre-trained client models is M=2, RA-FTL generally outperforms standard FTL by demonstrating faster early convergence and higher accuracy across different levels of heterogeneity. The fastest convergence time is 18, and the highest accuracy reaches 99.18%. This advantage is particularly pronounced in highly heterogeneous scenarios with α values of 0.1 and 0.5. Since the pre-trained client models selected possess superior data representation capabilities, RA-FTL achieves a higher final accuracy than standard FTL. Under weaker non-IID conditions, RA-FTL converges faster than standard FTL and significantly faster than FL. It also improves accuracy by 2–3%, approaching the maximum level of 100%. Under low to moderate heterogeneity, the deeper the frozen layer, the better

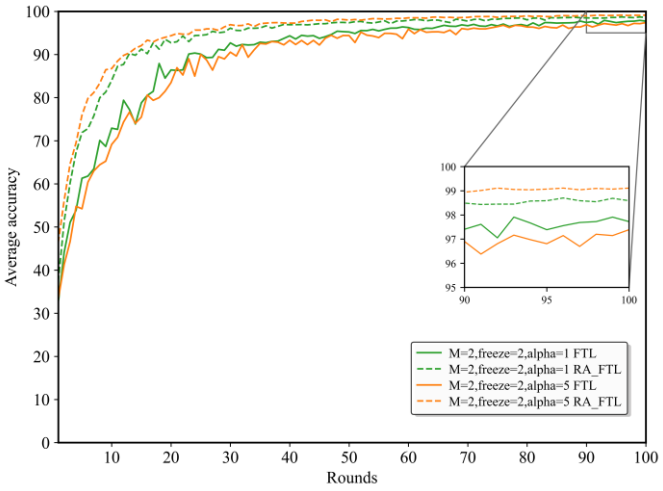

Fig. 18 Comparison between RA-FTL and standard FTL at α=1 and 5 with freezing 2 layers.

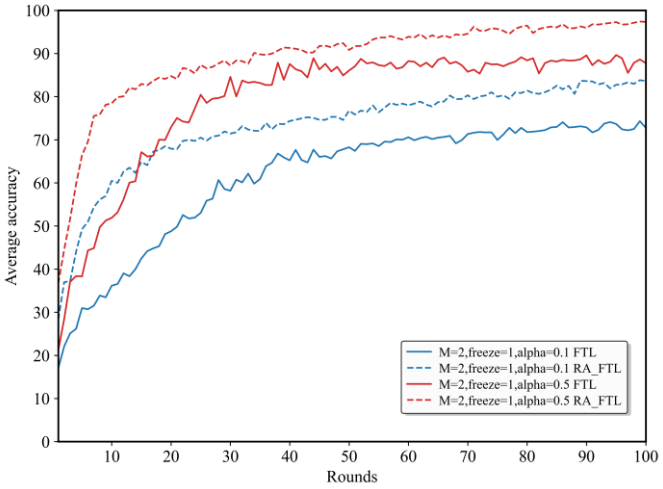

Fig. 19 Comparison between RA-FTL and standard FTL at α=0.1 and 0.5 with freezing 1 layer.

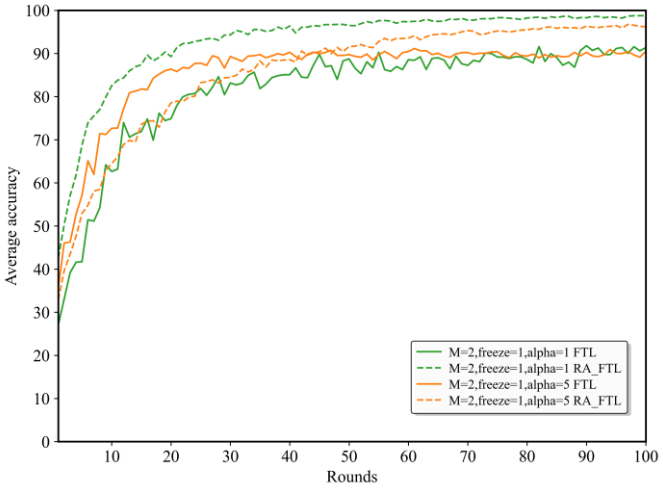

Fig. 20 Comparison between RA-FTL and standard FTL at α=1 and 5 with freezing 1 layer.

the model performance. This further demonstrates that the pre-training mechanism in a competitive environment can effectively select clients with better data representation capabilities. However, in strong non-IID scenarios, a shallower RA-FTL is preferable. Compared to traditional FL and FTL, although we use more clients for pre-training than FTL, the faster convergence speed of RA-FTL significantly reduces communication overhead. Under identical conditions, achieving the same accuracy can result in a reduction in communication overhead of up to 55.6%. It should be noted that RA-FTL's competitive mechanism introduces more pre-training clients, thereby increasing communication overhead. In strongly heterogeneous scenarios, this overhead is largely offset by the convergence advantage; however, in weakly heterogeneous scenarios, where the FTL model already performs well, RA-FTL's communication overhead is amplified. Similarly, due to its mechanism, this approach increases convergence speed and accuracy by adding pre-trained clients—is limited to a small number of pre-trained FTL, which aligns well with real-world massive-scale federated systems.

TABLE II
THE COMPARISON OF COMMUNICATION OVERHEAD BETWEEN RA-FTL AND STANDARD FTL AND FL WHEN = 0.1

Algorithm	RA-FTL (all)	FTL (all)	RA-FTL (first)	FTL (first)	FL
Average convergence iterations	70	71	57	66	65
Average convergence accuracy	67.40± 2.71%	59.50± 12.25%	75.7± 4.84%	70.49± 1.66%	57.55± 29.66%
Convergence communication overhead	366.18 MB	371.13 MB	300.24 MB	347.74 MB	342.92 MB
Final convergence accuracy	71.45± 5.25%	64.80± 11.40%	83.63± 4.11%	72.76± 2.60%	76.16± 2.27%

TABLE III
THE COMPARISON OF COMMUNICATION OVERHEAD BETWEEN RA-FTL AND STANDARD FTL AND FL WHEN α = 0.5

Algorithm	RA-FTL (all)	FTL (all)	RA-FTL (first)	FTL (first)	FL
Average convergence iterations	24	45	32	59	52
Average convergence accuracy	90.95± 3.32%	91.76± 2.37%	92.02± 2.09%	86.84± 6.17%	77.57± 27.69%
Convergence communication overhead	126.97 MB	234.74M B	170.07M B	307.06M B	272.95M B
Final convergence accuracy	97.66± 1.59%	96.79± 0.51%	97.33± 2.47%	87.77± 5.44%	89.50± 15.31%

TABLE IV
THE COMPARISON OF COMMUNICATION OVERHEAD BETWEEN RA-FTL AND STANDARD FTL AND FL WHEN $\alpha = 1$

Algorithm	RA-FTL (all)	FTL (all)	RA-FTL (first)	FTL (first)	FL
Average convergence iterations	23	33	26	61	62
Average convergence accuracy	93.05± 1.33%	91.24± 2.72%	92.98± 1.30%	89.24± 3.46%	91.94± 2.38%
Convergence communication overhead	119.62 MB	173.11 MB	135.42 MB	318.87 MB	324.40 MB
Final convergence accuracy	98.58± 0.79%	97.72± 0.68%	98.76± 0.85%	91.33± 6.07%	97.46± 0.61%

TABLE V
THE COMPARISON OF COMMUNICATION OVERHEAD BETWEEN RA-FTL AND STANDARD FTL AND FL WHEN $\alpha = 5$

Algorithm	RA-FTL (all)	FTL (all)	RA-FTL (first)	FTL (first)	FL
Average convergence iterations	23	40	39	29	72
Average convergence accuracy	93.84± 1.26%	91.46± 2.53%	89.47± 3.88%	88.56± 3.49%	92.15± 1.01%
Convergence communication overhead	119.62 MB	208.52 MB	202.61 MB	150.90 MB	377.93 MB
Final convergence accuracy	99.11± 0.08%	97.38± 0.35%	96.20± 2.34%	90.49± 11.74%	96.73± 1.37%

VI. Conclusion

This paper proposes a resource-aware federated transfer learning framework, RA-FTL, for non-independent and identically distributed (non-IID) federated learning tasks with heterogeneous client resources. Specifically, in scenarios with a limited number of pre-trained models, a synchronous pre-training competitive selection mechanism is employed to select the optimal feature extraction layer from a proxy validation set for transfer and federated learning. Experiments conducted on the MSTAR dataset led to the following conclusions:

1. In IID-biased scenarios, FTL demonstrates a significant performance gain over FL, though this gain exhibits stochastic fluctuations.

2. Under the assumption of strong non-independent and identically distributed (non-IID) data, if the distribution of data from the pre-training client does not represent the global distribution, the learned feature extractor will be biased; freezing too many layers will limit the subsequent client's ability to adapt to local data.

3. In highly heterogeneous scenarios, FTL exhibits an optimal range for the number of pre-trained clients; more is not necessarily better. This optimal range is often near the lower end of the spectrum; in our experiments, two pre-trained clients yielded the best results.

4. By employing resource-aware competitive pre-training, feature extraction layers with strong data representativeness can be obtained. Subsequent federated main training of the model achieves faster convergence and higher accuracy. Furthermore, due to this rapid convergence, the volume of communication traffic is significantly reduced.

References

[1] N. C. Luong et al., "Advanced Learning Algorithms for Integrated Sensing and Communication (ISAC) Systems in 6G and Beyond: A Comprehensive Survey," in IEEE Communications Surveys & Tutorials, vol. 28, pp. 2572-2611, 2026.

[2] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in Proc. 20th Int. Conf. Artif. Intell. Statist. (AISTATS), Fort Lauderdale, FL, USA, Apr. 2017, pp. 1273–1282.

[3] B. Zhang and M. R. Shikh-Bahaei, "Federated learning enhancement through transfer and continual learning integration: Analyzing effects of different levels of Dirichlet distribution," Wireless World Research and Trends Magazine, pp. 65–70, 2024.

[4] T. Wu, X. Li, P. Gao, W. Yu, L. Xin and M. Guo, "Resource-Aware Personalized Federated Learning Based on Reinforcement Learning," in IEEE Communications Letters, vol. 29, no. 1, pp. 175-179, Jan. 2025.

[5] Y. Wan, Y. Qu, W. Ni, Y. Xiang, L. Gao and E. Hossain, "Data and Model Poisoning Backdoor Attacks on Wireless Federated Learning, and the Defense Mechanisms: A Comprehensive Survey," in *IEEE Communications Surveys & Tutorials*, vol. 26, no. 3, pp. 1861-1897, thirdquarter 2024.

[6] S. P. Patil and R. Kaushik, "A Federated Transfer Learning Framework with Differential Privacy for Secure Monkeypox Diagnosis," *2025 IEEE International Students' Conference on Electrical, Electronics and Computer Science (SCEECS)*, Bhopal, India, 2025, pp. 1-5.

[7] C. Yu, S. Shen, S. Wang, K. Zhang and H. Zhao, "Communication-Efficient Hybrid Federated Learning for E-Health With Horizontal and Vertical Data Partitioning," in *IEEE Transactions on Neural Networks and Learning Systems*, vol. 36, no. 3, pp. 5614-5628, March 2025.

[8] Y. Chen, X. Sun and Y. Jin, "Communication-Efficient Federated Deep Learning With Layerwise Asynchronous Model Update and Temporally Weighted Aggregation," in *IEEE Transactions on Neural Networks and Learning Systems*, vol. 31, no. 10, pp. 4229-4238, Oct. 2020.

[9] C. Wu, F. Wu, L. Lyu, Y. Huang, and X. Xie, "Communication-efficient federated learning via knowledge distillation," *Nature Communications*, vol. 13, Art. no. 2032, 2022.

[10] Sa Xiao, Xiaoge Huang, Xuesong Deng, Bin Cao, and Qianbin Chen, "DB-FL: DAG blockchain-enabled generalized federated dropout learning," *Digital Communications and Networks*, vol. 11, no. 3, pp. 886–897, 2025.

[11] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, "Federated optimization in heterogeneous networks," in *Proc. 3rd Conf. Mach. Learn. Syst. (MLSys)*, 2020.

[12] B. Zhang and M. Shikh-Bahaei, "Improving Federated Transfer Learning for Different Levels of Non-IID Data with Multi-Round Transfer and Forgetting Mitigation," *ICC 2025 - IEEE International Conference on Communications*, Montreal, QC, Canada, 2025, pp. 1584-1589.

[13] F. Sattler, S. Wiedemann, K.-R. Müller, and W. Samek, "Robust and communication-efficient federated learning from non-IID data," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 31, no. 9, pp. 3400–3413, 2020.

[14] J. R. Diemunsch and J. Wissinger, "Moving and stationary target acquisition and recognition (MSTAR) model-based automatic target recognition: search technology for the standard operating procedure," in *Proc. SPIE, Algorithms for Synthetic Aperture Radar Imagery V*, vol. 3370, Orlando, FL, USA, 1998, pp. 481–492.

[15] H. Peng, C. Wu, and Y. Xiao, "FD-IDS: Federated learning with knowledge distillation for intrusion detection in non-IID IoT environments," *Sensors*, vol. 25, no. 14, Art. no. 4309, 2025.